ReTally Expiry / Food-Waste Alerts — Technical Design

Feature: "You bought spinach on Oct 5 — it goes bad in 2 days. Use it tonight." **Status:** Design (awaiting Mukesh's approval on open questions) **Date:** 2026-10-08 **Stack:** Kotlin Multiplatform (shared) · SQLDelight 2.1.0 · Firebase (retally-c5212, us-east1) · local notifications

---

1. USDA FoodKeeper — Research Findings

What it is

The USDA Food Safety and Inspection Service (FSIS) publishes the **FoodKeeper** dataset — shelf-life guidance for 600+ foods, with separate timelines for **refrigerator, freezer, and pantry**. It was built with Cornell University and the Food Marketing Institute. This is the authoritative US source; multiple indie apps already bundle it.

Availability (verified 2026-10-08)

Question	Answer
Public API?	**No.** One GitHub doc references `foodkeeper.usda.gov/api/products`, but the host is unreachable (connection fails). No documented public API exists.
Downloadable dataset?	**Yes.** Data.gov: `catalog.data.gov/dataset/fsis-foodkeeper-data` — JSON + XLS, English/Spanish/Portuguese. Direct JSON: `http://www.fsis.usda.gov/shared/data/EN/foodkeeper.json` (note: fsis.usda.gov is bot-walled; curl gets "Access Denied" — a browser or the XLS mirror works, or use a GitHub mirror).
License	**CC0 1.0 (public domain).** Free to bundle commercially.
Freshness	Last updated **2025-01-22**; prior update 2020. Updates are **irregular, years apart**.
Size	**661 raw products → 613 transformable** (48 have no shelf-life data, 7.3%).

Raw schema (per product)

Each product has: `name`, `name_subtitle`, `keywords`, plus per storage condition a `{min, max, metric}` triple. Storage conditions come in two flavors:

- **DOP_*** = "date of purchase" (unopened, counting from purchase) — `DOP_Refrigerate`, `DOP_Pantry`, `DOP_Freeze`
- **Opened** variants — `Refrigerate`, `Pantry`, `Freeze` (after opening)

Metrics: Days / Weeks / Months (= 30d) / Years (= 365d). Qualitative values exist too: "Indefinitely", "Not Recommended", "When Ripe", "Hours" (8 products) — these need special handling or exclusion.

For ReTally, the DOP_ (date-of-purchase) variants are the correct ones — we count from the receipt's purchase date, and the item is unopened at that point.

Cleaned/transformed shape (what we'd bundle)

A community-cleaned version (613 items, CC0) looks like this — this is the practical format:

```
{
  "name": "Lettuce (leaf, spinach)",
  "cat": "Produce / Fresh Vegetables",
  "kw": ["lettuce", "leaf", "spinach"],
  "pantry": null,
  "fridge": [3.0, 7.0],
  "freezer": null
}
```

- `fridge` / `pantry` / `freezer`: `[min_days, max_days]` or `null`
- 24 categories; **1,268 unique keywords** across the set
- Coverage: fridge 363 items · pantry 294 · freezer 339
- **Bundled size: ~106 KB JSON** — trivial for an app bundle.

Data-quality caveats (found during research)

- Some entries have suspicious gaps (e.g., "Milk (plain or flavored)" shows `fridge: null` in one community transform — fresh milk obviously refrigerates). **Mitigation:** do our own transform from raw USDA JSON with the DOP-priority chain (`DOP_Refrigerate → Refrigerate → DOP_Pantry → Pantry → DOP_Freeze → Freeze`), and layer **hand-tuned overrides for the top ~40 grocery staples** (milk, eggs, bread, chicken…) as the highest-priority lookup.
- 48 products have no usable data → they fall through to category defaults.
- Shelf lives are guidelines ("some foods may spoil more quickly"), not guarantees — the UI copy should reflect this ("usually good for ~5 days").

Precedents (how others did it)

- **smartpantry** (open source): bundles 613-item FoodKeeper JSON, precision-first keyword matcher (two independent keyword hits, exact head-keyword hit, or rare-keyword hit), expiry

= purchase date + span midpoint, notify window = `max(2 days, 20% of span)`. Also learns per-item cadence from purchase history.

- **renovate**: hand-tuned specifics → category keywords → FoodKeeper match → 14-day default.
- **whats-in-the-fridge**: per-item FoodKeeper lookup by name, sliders as fallback.

---

2. Item Name Matching Strategy

Receipt OCR gives messy names: `"1921532 DOSABATTER"`, `"ORG SPINACH 5OZ"`, `"3 WHOLE MILK"`. The matcher runs **on-device, in shared KMP code**, at scan-save time.

Pipeline

```
raw OCR name
  → normalize()
  → hand-tuned staples map        (highest priority, ~40 entries)
  → FoodKeeper keyword match      (TF-IDF-lite scoring, threshold)
  → ReTally category default
  → global default (14 days)
```

Step 1 — normalize()

1. Uppercase.
2. Strip leading store item codes: `^\d+\s+` ("1921532 DOSABATTER" → "DOSABATTER").
3. Strip quantity/weight tokens: `\d+(\.\d+)?\s?(OZ|LB|G|KG|ML|L|CT|PK|EA|PT|QT|GAL)\b` ("ORG SPINACH 5OZ" → "ORG SPINACH").
4. Strip leading quantity words: `^\d+\s+` ("3 WHOLE MILK" → "WHOLE MILK").
5. Drop organic/store-brand markers that don't change shelf life: `ORG`, `ORGANIC`, `KS`, `KIRKLAND` (keep a flag, don't let them break matching).
6. Collapse whitespace, trim.

Step 2 — hand-tuned staples (~40 entries, curated once, bundled)

The most-bought items get exact answers, immune to USDA gaps:

Item	Days	Storage
milk	7	fridge
eggs	28	fridge
bread	5	pantry

Item	Days	Storage
spinach / lettuce / bagged greens	5	fridge
chicken (raw)	2	fridge
ground beef	2	fridge
yogurt	14	fridge
cheese (hard)	180	fridge
bananas	4	pantry
apples	30	fridge
…	…	…

Step 3 — FoodKeeper keyword match (TF-IDF-lite)

- Each FoodKeeper entry has a keyword list (`kw`). Precompute **document frequency** per keyword at data-build time and store it in the bundled JSON.
- Score = Σ `log(totalItems / docFreq(kw))` over keywords found as substring/phrase in the normalized name. Rare keywords ("arugula") score higher than common ones ("cheese").
- **Accept if:** score ≥ threshold (tune ~2.0) **and** at least one of: (a) two independent keyword hits, (b) exact head-keyword hit (first keyword = full normalized name), or (c) a rare keyword hit (docFreq ≤ 3). This mirrors smartpantry's precision-first rule and avoids "cream" matching everything with "cream" in it.
- On match: `shelfLifeDays = round((min+max)/2)` (span midpoint), storage = first available of [fridge, pantry, freezer] biased by category (produce/dairy/meat → fridge; frozen → freezer; else pantry-first).

Step 4 — category defaults

ReTally already categorizes every item (`ReceiptItem.categoryId`, server keyword rules). Map:

Category	Days	Storage
produce	5	fridge
dairy	7	fridge
meat / poultry / seafood	3	fridge
deli / prepared	4	fridge
bakery	5	pantry

Category	Days	Storage
frozen	180	freezer
pantry / canned / dry goods	365	pantry
beverages	30	pantry
grocery (default bucket)	14	fridge

Step 5 — global default: 14 days, fridge.

Every item gets *an* answer; the UI shows "~" for estimated dates.

Expected hit rate

Top-40 staples cover ~60-70% of grocery line items by frequency; keyword match should resolve most of the rest. Category/global fallback guarantees 100% coverage with graceful degradation.

---

3. Client-Side Architecture (Kotlin Multiplatform, on-device)

3.1 Database — new `item_expiry` table (SQLDelight, `retally.db`)

```
CREATE TABLE item_expiry (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    receipt_id      TEXT NOT NULL,     -- FK-ish to receipt row (receipts are the source of truth)
    item_name       TEXT NOT NULL,     -- raw OCR name, for display
    normalized_name TEXT NOT NULL,     -- post-normalize(), for grouping
    category_id     TEXT NOT NULL,     -- ReTally category at scan time
    purchase_date   INTEGER NOT NULL,  -- epoch millis (receipt.purchaseDate)
    expiry_date     INTEGER NOT NULL,  -- epoch millis
    shelf_life_days INTEGER NOT NULL,  -- the estimate used
    storage         TEXT NOT NULL,     -- 'fridge' | 'pantry' | 'freezer'
    source          TEXT NOT NULL,     -- 'staple' | 'foodkeeper' | 'category' | 'default'
    notified        INTEGER NOT NULL DEFAULT 0,  -- expiring-soon notification sent
    dismissed       INTEGER NOT NULL DEFAULT 0,  -- user dismissed
    created_at      INTEGER NOT NULL
);
CREATE INDEX idx_item_expiry_date ON item_expiry(expiry_date);
CREATE INDEX idx_item_expiry_norm ON item_expiry(normalized_name, purchase_date DESC);
```

Why a separate table (not JSON on receipt): expiry rows need date-indexed queries ("what expires in the next 3 days?") and per-row notification flags — exactly what SQLDelight is for. (Same rationale as the earlier `receipt` / `known_store` / `budget` migrations.)

Schema migration: bump SQLDelight version; `CREATE TABLE IF NOT EXISTS` in the migration path. Existing installs get the table on upgrade; no data backfill (only new scans generate rows — acceptable for MVP).

3.2 Shelf-life data bundle (on-device)

- **File:** `shared/src/commonMain/composeResources/files/shelf_life.json` (~106 KB) — read via `Res.readBytes("files/shelf_life.json")`, parsed once with kotlinx.serialization into an in-memory index.
- **Contents:** cleaned FoodKeeper entries + `docFreq` per keyword + `version` field (e.g. `"2025-01-22"`) + the hand-tuned staples map + category defaults. One file, one parse, ~50ms.
- **Why bundled, not server-fetched (MVP):** USDA updates years apart; 106 KB is negligible; lookup must work offline (scans happen in store basements). A `version` field leaves the door open for server deltas later (see §4).

3.3 Calculation timing — at scan-save

In `ScanReviewContract` persist flow (already `runCatching`-guarded), after the receipt row is written:

```
for each item in receipt.items:
    norm = normalize(item.name)
    (days, storage, source) = lookupShelfLife(norm, item.categoryId)
    expiry = receipt.purchaseDate + days * 24h
    item_expiry.insert(...)
```

- Runs in a background coroutine; failure never blocks the save (log + continue).
- **Re-purchase = new row.** Never overwrite: the old spinach may still be in the fridge. Notification scheduler (below) only considers the **latest purchase per normalized_name**, so weekly milk buys don't spam — older rows stay for history/AI.
- Items with `shelf_life_days > 30` (pantry/frozen) get rows but are excluded from notifications.

3.4 Notification scheduling

Trigger rule (adopted from smartpantry, validated in the wild): **Notify when** `daysUntilExpiry ≤ max(2, 20% of shelfLifeDays)` — e.g. spinach (5d) → 2 days before; eggs (28d) → ~6 days before.

- **iOS:** `UNUserNotificationCenter` with `UNTimeIntervalNotificationTrigger` at `(expiryDate - window)`. Limit: **64 pending requests/app** — fine, since we only schedule for distinct items expiring within ~14 days (latest-purchase-per-item). Reconcile on every app launch + after every scan-save (cancel stale, schedule new). Follows the existing `NearbyStoreMonitor` / `DebugLocationPing` UNUserNotificationCenter pattern.

- **Android:** WorkManager `OneTimeWorkRequest` with `setInitialDelay` → fires a notification via the existing notification channel pattern (`NearbyNotifications`). Inexact timing is fine (a few hours' drift doesn't matter for a 2-day warning); WorkManager survives reboots and is battery-friendly. No `SCHEDULE_EXACT_ALARM` permission needed.
- **Copy:** `"Spinach goes bad in 2 days" / "You bought it Oct 5 — use it tonight."` Deep-link tap → receipt detail.
- **Dismissal:** "Used it" action on the notification/nudge → sets `dismissed=1`, cancels siblings for that item.

3.5 Nudge-engine integration

Add a first-class nudge type — this is the highest-leverage UI, zero new screens for MVP:

```
Nudge.ExpiringSoon(itemName: String, daysLeft: Int, receiptId: String)
```

- Fires when `0 ≤ daysLeft ≤ 3`, priority below `DoubleBought`, above `SavingsRecap` (tune later).
- Renders in the existing `NudgeCard` / `NudgeCarousel` with a 🥬-style treatment and "Got it / Used it" actions.
- 24h per-item cooldown (reuse the nudge engine's existing cooldown infra).

3.6 UI surface (MVP)

1. **Nudge card on Home** (above) — primary surface, no new navigation.
2. **Receipt detail:** per-item "Expires Oct 12" / "~5 days" chip on perishable rows (uses the `item_expiry` row for that receipt+item).
3. *(Future, not MVP)* Dedicated "Freshness" screen / tab listing everything by expiry.

3.7 Koin registrations (shared `di/Koin.kt`)

```
single { ShelfLifeRepository(json = get(), settings = get()) }   // bundled data + version
single { ExpiryRepository(db = get()) }                          // item_expiry CRUD
single { ExpiryScheduler(expiryRepo = get(), notifier = get()) } // platform notifier via
expect/actual
```

Platform `LocalNotifier` expect/actual: Android → WorkManager; iOS → UNUserNotificationCenter. (Mirrors the existing `PlatformGeocoder` expect/actual pattern.)

---

4. Server-Side Architecture (Firebase, retally-c5212, us-east1)

Recommendation: thin server, fat client (MVP)

The server is a **data pipe + version gate**, not a compute layer — matching the existing `getCategories` / `getStoreIcons` pattern Mukesh already approved (Firestore console-editable, no redeploy).

4.1 Firestore — `config/shelfLife`

One document (console-editable, like `config/itemCategories`):

```
config/shelfLife {
  version: "2025-01-22",          // bump when data changes
  items: [                        // full snapshot, ~613 entries
    { name, keywords[], fridgeDays:[min,max], pantryDays, freezerDays,
      category, storageHint }
  ],
  staples: { "milk": {days:7, storage:"fridge"}, ... },  // hand-tuned overrides
  categoryDefaults: { "produce": {days:5, storage:"fridge"}, ... }
}
```

4.2 Cloud Function — `getShelfLife` (2nd-gen, TypeScript, us-east1)

```
// POST {} → { ok:true, version, items?, staples?, categoryDefaults? }
// Client sends its bundled version via If-None-Match style param { clientVersion }.
// If clientVersion == version → { ok:true, version, upToDate:true } (empty payload).
// Else → full snapshot. (Delta encoding is overkill at 106 KB.)
```

- App fetches at launch, cached silently (same pattern as categories: Settings keys `shelfLife.version`, `shelfLife.data.v1`; failures silent).
- **Seed:** one-time Node script transforms raw USDA JSON → Firestore doc (reuse the DOP-priority transform; script lives in `functions/scripts/seedShelfLife.ts`, run once, kept for re-seeds).

4.3 Why not server-side expiry calculation?

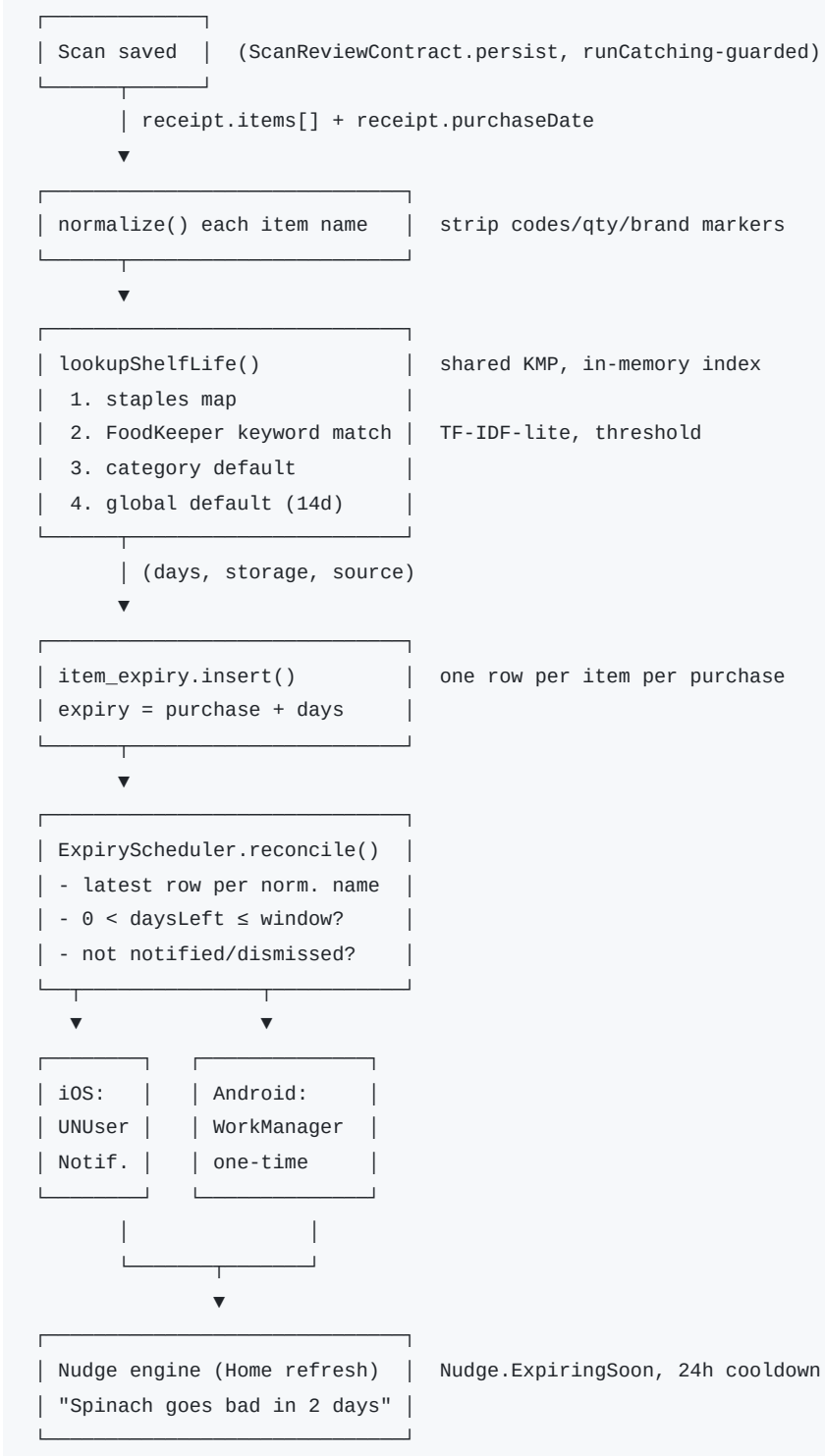
- **Offline-first:** scans happen in stores with dead zones; the save flow must not depend on network.
- **Latency:** notification scheduling must happen synchronously at save.
- **Cost:** per-scan function invocations for pure lookups = waste.
- The server's job is keeping the *reference data* fresh, which changes every few years.

4.4 Future ML — `predictExpiry` (not MVP)

Once purchase history is rich: a function (or on-device model) that learns **per-item consumption cadence** — "Mukesh buys milk every 7 days → his effective milk life is 7 days, regardless of USDA's 7–14." Inputs: `{ normalizedName, categoryId, purchaseIntervals[], usdaDays }` → `{ days, confidence }`. This is the durable moat: USDA gives the prior, behavior gives the posterior. On-device learning keeps it private; no PII leaves the phone.

---

5. Data Flow



```
Re-purchase ⟶ new row; scheduler notifies on latest only; old rows → history/AI.
User "Used it" ⟶ dismissed=1; cancel pending; nudge resolved.
```

---

6. MVP Scope vs Future

MVP (ship it)

- One-time USDA transform script → `shelf_life.json` (613 items + staples + category defaults), bundled in `composeResources/files/`
- `normalize()` + 4-level `lookupShelfLife()` in shared KMP
- `item_expiry` SQLDelight table + migration
- Expiry calc at scan-save (guarded, background)
- `ExpiryScheduler` + platform notifiers (iOS UNUserNotificationCenter / Android WorkManager)
- `Nudge.ExpiringSoon` in the nudge engine + Home card actions ("Used it")
- Receipt-detail "Expires Oct 12" chips
- Koin wiring, string resources (en only)
- Server: `config/shelfLife` seed + `getShelfLife` function (version-gated fetch; app works fully without it)

Future (post-MVP)

- Dedicated Freshness screen (all items sorted by expiry, storage filter)
- Per-item storage override ("I froze the chicken" → recompute from freezer days)
- Learned shelf life from repurchase cadence (on-device; later `predictExpiry`)
- "Use-it-up" recipe nudges ("3 items expiring → stir-fry tonight")
- Household sharing of expiry state
- Community data: anonymized "most users toss X after Y days" refinement
- Spanish/Portuguese FoodKeeper bundles (data already exists)

Deliberately out

- Barcode/UPC-based lookup (receipt OCR has no barcodes reliably)
- Photo-based spoilage detection (CV freshness models exist in academia but are unreliable in production)
- Server-side per-scan calculation (offline + cost, §4.3)

---

7. Open Questions for Mukesh

1. **Bundled vs server-first:** MVP bundles the 106 KB snapshot (works offline, zero server dependency). The `getShelfLife` function keeps it updatable without app releases. OK with bundled-first?
2. **Notification timing:** Proposing notify at `max(2 days, 20% of shelf life)` before expiry, plus an `ExpiringSoon` nudge at ≤3 days. Too chatty? Want a Settings toggle ("Expiry alerts on/off")?
3. **Storage assumption:** Default to fridge for perishables without asking. Add a per-item "I froze it" override later, or in MVP?
4. **Re-purchase semantics:** New row per purchase, notify on latest only. If he buys spinach twice in one week, only the newest triggers. Good?
5. **"Used it" UX:** Dismiss from the nudge/notification. Should dismissed items also disappear from receipt-detail chips, or stay with a ✓?
6. **Scope of items:** All receipt items get expiry rows (even pantry 365-day ones, for future AI), but only ≤30-day items notify. OK?
7. **Copy tone:** "Spinach goes bad in 2 days — use it tonight." Fine, or softer ("might be going bad")? USDA data is a guideline, not a guarantee — the "~" and "usually" wording matters for trust.
8. **Plus gating?** Expiry alerts feel like a core free feature (traction driver). Keep free, or reserve the Freshness screen / learned predictions for Plus?

Appendix: Key References

- Dataset: `catalog.data.gov/dataset/fsis-foodkeeper-data` (CC0, updated 2025-01-22)
- Raw JSON (bot-walled, use browser/XLS mirror): `http://www.fsis.usda.gov/shared/data/EN/foodkeeper.json`
- Transform precedents: smartpantry (613-item JSON + keyword matcher), chenda (661→613 transform, DOP-priority chain), renovate (587 items)
- Shelf-life estimate convention: **span midpoint**; notify window: **max(2 days, 20% of span)**
- Platform notes: iOS 64-pending-notification limit; Android WorkManager one-time w/ delay (no exact-alarm permission needed)